

Drowsy Bot

Drowsy Bot is a Discord bot focused on three areas:

- hosted stage and queue events
- invite-link moderation and invite allowlisting
- OBS/browser overlays for now-singing and sponsor/ad display

It is built with Node.js and discord.js and stores runtime data in JSON files under `data/`.

Current Feature Set

- Multi-stage queue flow for voice events
- Intermission radio playback from a local MP3 file
- Public scheduled-event lookup
- Staff announcement command for posting through the bot
- Invite moderation with allowlist and cleanup tools

Removed Systems

These systems are no longer part of the bot:

- reaction roles
- logging system
- moderation commands
- sticky-role handling

Requirements

- Node.js 20 or newer
- A Discord application with a bot user
- A Discord server where the bot can be invited and managed

Installation

```
git clone https://github.com/ABZ-Digital-Group/drowsy_bot.git
cd drowsy_bot
npm install
node index.js
```

Environment Variables

Create a `.env` file:

```
DISCORD_TOKEN=your_bot_token
CLIENT_ID=your_application_id
GUILD_ID=your_server_id
ALLOW_INVITE_PASSWORD=optional_dm_password
OBS_HTTP_PORT=8080
OBS_HTTP_HOST=0.0.0.0
```

Variable Reference

- `DISCORD_TOKEN` : bot token from the Discord developer portal
- `CLIENT_ID` : Discord application ID used to register guild commands
- `GUILD_ID` : guild where slash commands are registered
- `ALLOW_INVITE_PASSWORD` : optional password used by the DM command `!allowinvite <password>`
- `OBS_HTTP_PORT` : optional port for the built-in OBS overlay endpoint
- `OBS_HTTP_HOST` : optional bind host for the OBS overlay endpoint

Discord Intents

The bot uses these intents:

- Guilds
- GuildMessages
- MessageContent
- GuildVoiceStates
- GuildMembers
- DirectMessages
- GuildScheduledEvents

Staff Access Model

Staff-only commands are available to:

- the guild owner
- members with `Administrator`
- members with `Manage Guild`
- members with `Moderate Members`
- members who hold one of these role names:
 - `Guards`
 - `Knights`
 - `Drowsy Defenders`
 - `God`

Project Structure

```
index.js
src/
  commands.js
  config.js
  helpers.js
  state.js
  features/
    community.js
    stage.js
assets/
data/
```

Data Files

The bot creates and uses these files under `data/` :

- `guild-config.json` : guild-level bot settings
- `allowed-invite-users.json` : invite allowlist

Some older data files may still exist from previous versions, but they are no longer used by the current runtime.

Stage Queue System

The stage queue is built for hosted performances or open-mic style events.

Commands

- `/queue`
- `/start-queue`
- `/open-queue`
- `/close-queue`
- `/stop-queue`
- `/next`
- `/radio`

How It Works

1. A staff member joins the voice channel they want to host.
2. They run `/start-queue` in a text channel.
3. The bot posts a queue panel with buttons.
4. Staff can run `/start-queue` in additional text channels for the same active voice channel to create mirrored control panels.
5. Members join or leave the lineup using buttons from any active control panel.
6. If nobody is currently up, the first person to join is moved on stage immediately.
7. Staff move the event forward with `/next` or the active speaker ends their turn with `Done` .
8. Anyone can run `/queue` to post the current lineup in the channel.
9. Staff can close the queue to new joiners with `/close-queue` and reopen it with `/open-queue` .

Queue Buttons

- `Join Queue`
- `Leave`
- `Done`
- `Staff: Close Queue / Staff: Open Queue`

Notes

- `assets/intermission.mp3` is used for radio playback.
- `assets/obs-now-singing.txt` is updated with the current singer name for OBS text sources.
- only one voice channel can be active per server at a time
- multiple text-channel control panels can manage that same active voice channel
- uploaded sponsor images are stored under `assets/ads/` for the OBS ad overlay
- when a stage is active, the bot also posts the current sponsor ad into the active control-panel text channels and updates it as ads change or rotate

Shy Stage Overflow Rooms

- voice channels named `Shy Stage 1` , `Shy Stage 2` , `Shy Stage 3` , and so on are managed automatically

- Roman numeral names are also supported, such as `shy stage I` , `shy stage II` , and `Shy Stage III`
- `Shy Stage 1` and `Shy Stage 2` stay visible at all times
- when the first non-bot member joins the last currently available shy stage, the bot creates or reveals the next shy stage channel and sets its member limit to `3`
- empty overflow rooms such as `Shy Stage 3+` are hidden again when everyone leaves
- the bot posts a side-chat notice to the first text channel in the same category when a shy stage room opens

How It Works

1. Keep two base shy stages available at all times.
2. A member joins the last currently available shy stage.
3. If that join makes the room active with its first non-bot member, the bot prepares the next shy stage.
4. The next shy stage is created if it does not exist yet, or kept hidden and ready if it already exists.
5. The bot sets the shy stage user limit to `3` .
6. The bot sends a notice in the first text channel under the same category.
7. When an overflow shy stage becomes empty again, it is hidden from members until it is needed next time.

Example Flow

1. `Shy Stage I` and `Shy Stage II` are visible.
2. Someone joins `Shy Stage I` . Nothing new is created yet because `Shy Stage II` is still the last available shy stage.
3. Someone joins `Shy Stage II` .
4. Because `Shy Stage II` is the last currently available shy stage and just got its first non-bot member, the bot creates or reveals `Shy Stage III` .
5. `Shy Stage III` stays empty and hidden until needed, then becomes the next overflow target.
6. Once everyone leaves `Shy Stage III` , it is hidden again.

Requirements

- the bot must be able to view the shy-stage channels
- the bot must have permission to manage channels in that category
- the bot must be able to send messages in the side-chat text channel

OBS Text Source

If the bot and OBS run on the same machine, you can add a text source that reads from:

```
assets/obs-now-singing.txt
```

The bot keeps that file updated with:

- the current singer's display name
- `Open Mic` when nobody is up
- `Show Ended` when the queue is stopped

OBS Browser Source For VPS Hosting

If the bot is hosted on a VPS, OBS cannot read the bot's local filesystem directly. In that case:

1. Set `OBS_HTTP_PORT` in `.env` .
2. Open or proxy that port on the VPS.
3. In OBS, add a Browser Source pointing to:

```
http://YOUR_VPS_HOST:OBS_HTTP_PORT/obs/now-singing
```

That browser source now renders the active singer's server profile picture using their guild avatar when available, and falls back to their normal Discord avatar otherwise.

There is also a raw text endpoint available at:

```
http://YOUR_VPS_HOST:OBS_HTTP_PORT/obs/now-singing.txt
```

And a JSON endpoint with both the current display name and avatar URL:

```
http://YOUR_VPS_HOST:OBS_HTTP_PORT/obs/now-singing.json
```

The browser source updates whenever OBS refreshes the page. If you want near-live updates, set the Browser Source to refresh when it becomes active or use a short custom refresh workflow through OBS/browser-source controls.

OBS Advertisement Overlay

Staff can upload ad images directly through slash commands:

- `/ad-upload image:<attachment> [title]`
- `/ad-list`
- `/ad-show index:<number>`
- `/ad-rotate seconds:<number>`
- `/ad-rotate-stop`
- `/ad-remove index:<number>`

The active ad is exposed for OBS at:

```
http://YOUR_VPS_HOST:OBS_HTTP_PORT/obs/ad
```

There is also a JSON endpoint if you want to build your own browser source logic:

```
http://YOUR_VPS_HOST:OBS_HTTP_PORT/obs/ad.json
```

The ad browser source dims itself when no stage session is active and switches back on when the queue is running. When auto-rotation is enabled, the overlay advances through the uploaded ads on the interval you set.

Combined Live Event Screen

If you want one hosted page for a cloud VM or browser capture, use:

```
http://YOUR_VPS_HOST:OBS_HTTP_PORT/obs/live
```

That page combines the current singer card and the sponsor/ad panel into one layout and refreshes automatically.

There is also a matching JSON endpoint at:

```
http://YOUR_VPS_HOST:OBS_HTTP_PORT/obs/live.json
```

Events System

The bot can post active and upcoming Discord scheduled events.

Public Entry Points

- `/events`
- `-events`

Behavior

- fetches scheduled events from the guild
- filters to active and scheduled entries
- sorts by start time
- posts Discord event URLs so the client renders them naturally

Invite Moderation

Invite links are allowed only for the guild owner and users on the bot's invite allowlist.

Staff Commands

- `/allow-invites target:<user>`
- `/revoke-invites target:<user>`
- `/purge-invites [messages_per_channel]`

User Self-Allow Flow

Users can DM the bot:

```
!allowinvite your_password_here
```

If the password matches `ALLOW_INVITE_PASSWORD`, they are added to the allowlist.

Invite Cleanup

`/purge-invites` scans accessible text and announcement channels and deletes unauthorized invite links.

Command Reference

Public Commands

- `/events`
- `-events`

Staff Commands

- `/announce message:<text> [title] [color] [channel]` Use `\n` inside `message` if you want line breaks in the posted announcement.
- `/announce-color [color] [reset]`
- `/start-queue`
- `/stop-queue`
- `/next`
- `/radio`
- `/allow-invites`

- `/revoke-invites`
- `/purge-invites`
- `/ad-upload`
- `/ad-list`
- `/ad-show`
- `/ad-rotate`
- `/ad-rotate-stop`
- `/ad-remove`

First-Time Setup Checklist

1. Create the Discord application and bot.
2. Enable the required intents in the Discord developer portal.
3. Invite the bot to the target server.
4. Create `.env` with valid IDs and token.
5. Run `npm install`.
6. Add `assets/intermission.mp3` if you want radio playback.
7. Start the bot so it registers slash commands for the configured guild.
8. Test `/events` and the stage queue commands.

Deployment Notes

Typical update flow:

```
git pull origin main
npm install
node index.js
```

If you use `pm2` :

```
git pull origin main
npm install
pm2 restart drowsy_bot
```

Troubleshooting

Slash Commands Do Not Appear

Check:

- `CLIENT_ID` is correct
- `GUILD_ID` is correct
- the bot can log in successfully
- the bot has been restarted after deployment

Missing Dependency Error

If you see `Cannot find module 'dotenv'` or another dependency error, run:

```
npm install
```

Invite Moderation Does Not Delete Links

Check:

- the bot can manage messages in that channel
- the message actually matches the Discord invite regex
- the sender is not the guild owner
- the sender is not on the allowlist

Queue Radio Does Not Play

Check:

- `assets/intermission.mp3` exists
- the bot can connect and speak in the voice channel
- dependencies installed successfully on the host

Architecture Notes

- [index.js](#): bot bootstrap and event binding
- [src/config.js](#): constants, env vars, and file paths
- [src/state.js](#): file-backed persistence and shared runtime state
- [src/helpers.js](#): shared helper utilities
- [src/commands.js](#): slash command schema
- [src/features/stage.js](#): queue, speaker handoff, and radio flow
- [src/features/community.js](#): events and invite moderation

License

ISC